

Table of Contents

- 1. [Quick Overview](#)
- 2. [Firebase Setup](#)
- 3. [Authentication](#)
- 4. [Recipe Discovery – Home Page](#)
- 5. [Recipe Detail Page](#)
- 6. [Favourites](#)
- 7. [Create & Edit Recipe](#)
- 8. [User Profile](#)
- 9. [Admin Approval System](#)
- 10. [User-to-User Messaging](#)
- 11. [Chef AI Chatbot](#)
- 12. [Mobile Navigation](#)
- 13. [CSS Architecture](#)
- 14. [Firestore Data Model](#)
- 15. [How Everything Connects](#)

1. Quick Overview

Module	Files
Firebase Setup	<code>scripts/firebase.js</code>
Authentication	<code>scripts/auth.js</code> , <code>pages/Login.html</code> , <code>pages/Register.html</code>
Recipe Discovery	<code>scripts/home.js</code> , <code>Cookly.html</code>
Recipe Detail	<code>scripts/recipe_details.js</code> , <code>pages/recipe_details.html</code>
Favourites	<code>scripts/favourites.js</code> , <code>pages/favourites.html</code>
Create / Edit Recipe	<code>scripts/create_recipe.js</code> , <code>scripts/edit_recipe.js</code>
User Profile	<code>scripts/profile.js</code> , <code>pages/profile.html</code>
Admin Approval	<code>scripts/profile.js</code> (lines 298–444)
Messaging	<code>scripts/chat.js</code> , <code>pages/chat.html</code>
Chef AI Chatbot	<code>scripts/chatbot.js</code> , <code>styles/chatbot.css</code>
Mobile Nav	<code>scripts/navbar.js</code>

Module Summary

1. Firebase Setup – `scripts/firebase.js` Single responsibility file. Initializes the Firebase app, exports `auth`, `db` (Firestore), and `storage`. Also enables Firestore's persistent offline cache so the app works even with a slow connection. Every other JS file imports from here.

2. Authentication – `scripts/auth.js` + Login/Register HTML Four separate flows all in one file, each toggled by showing/hiding form divs:

Flow	What it does	Lines
Email login	<code>loginForm</code> submit → <code>signInWithEmailAndPassword</code>	131–165
Google login	<code>googleSignInBtn</code> click → <code>GoogleAuthProvider</code> popup	27–51
MFA challenge	After login, if resolver exists → verify TOTP code	54–88
Forgot password	Toggle hidden form → <code>sendPasswordResetEmail</code>	91–128
Registration	<code>registerForm</code> submit → <code>createUserWithEmailAndPassword</code> + <code>updateProfile</code>	168–223

3. Recipe Discovery — scripts/home.js + Cookly.html

Module	What it does	Lines
Data loading	<code>loadRecipes()</code> — switches between Firestore collections based on source filter	448–494
Filtering	<code>applyFilters()</code> — client-side filter by search, category, time, calories, sort order	418–446
Rendering	<code>renderPage()</code> — builds recipe cards with lazy vote loading and a "Load More" button	256–382
Voting	<code>handleCardVote()</code> — Firestore transaction to atomically increment/decrement counts	516–565
Favouriting	<code>toggleFavorite()</code> — writes/deletes doc in <code>users/{uid}/favorites/</code>	496–514
Auth UI sync	<code>updateNavbar()</code> — shows/hides buttons, welcome name, Messages badge	139–188

4. Recipe Detail Page — scripts/recipe_details.js

Module	What it does	Lines
Load recipe	<code>loadRecipeDetails()</code> — detects source from URL params, picks the right collection	346–381
Render recipe	<code>renderRecipe()</code> — builds the full card: hero image, summary, ingredients, instructions	63–107
Voting	<code>handleVote()</code> — same transaction pattern as home page	205–244
Comments	<code>loadComments()</code> / <code>submitComment()</code> / <code>deleteComment()</code>	247–343
Message author	<code>initInteractions()</code> — injects "Message Author" button for community recipes	110–173

5. Favourites — scripts/favourites.js

Module	What it does	Lines
Load favourites	Reads <code>users/{uid}/favorites/</code> , resolves each ID across 3 collections	129–160
Resolve recipe	<code>fetchRecipeById()</code> — tries <code>recipes</code> , then <code>published_recipes</code> , then <code>users/.../recipes_created</code>	113–127
Render cards	<code>recipeCard()</code> — builds card with an "x Remove" button	76–111
Unfavourite	<code>unfavorite()</code> — deletes the favourite doc and removes the card from DOM	63–74

6. Create & Edit Recipe — scripts/create_recipe.js + scripts/edit_recipe.js

Module	What it does	Lines
AI generation (photo)	Sends image as base64 to Gemini API, parses JSON from response	create_recipe.js 87–186
AI generation (text)	Sends text description to Gemini, same JSON parse	create_recipe.js 116–186
Fill form from AI	<code>fillFormFromRecipe()</code> — auto-populates all fields from Gemini's response	create_recipe.js 96–108
Form submit (create)	Validates, uploads image to Firebase Storage, writes to <code>users/{uid}/recipes_created</code>	create_recipe.js 202–271
Form submit (edit)	Same but uses <code>updateDoc</code> , preserves old image if unchanged	edit_recipe.js 91–144

7. User Profile — scripts/profile.js

Module	What it does	Lines

Load profile Module	loadProfile() — fetches auth user, counts favourites, loads created recipes What it does	249–295 Lines
Avatar upload	File input → Cropper.js modal → Firebase Storage upload → updateProfile(photoURL)	82–143
Inline name edit	Pencil toggle shows <input>, save calls updateProfile({displayName})	475–513
Inline email edit	Same pattern, calls verifyBeforeUpdateEmail()	516–553
Password reset	Sends reset email via sendPasswordResetEmail	556–583
2FA enable	Generates TOTP secret → shows QR code → verifies code → enrolls MFA	612–653
2FA disable	Re-authenticates → unenrolls the TOTP factor	655–708
Submit for review	submitForReview() — writes recipe to pending_reviews	206–246

8. **Admin Approval** — embedded in scripts/profile.js lines 298–444 Visible only when logged in as total_carnage24@yahoo.com. Loads pending_reviews, shows expandable rows, approve → publish, reject → revert to private.

9. **Messaging** — scripts/chat.js + pages/chat.html

Module	What it does	Lines
Encryption	getKey() derives AES-GCM key via PBKDF2 from convo ID + pepper	14–28
Encrypt/Decrypt	AES-GCM with random IV prepended to ciphertext	30–58
Start conversation	getOrCreateConvo() — deterministic ID (sorted UUIDs joined by _)	120–142
Load conversations	Real-time onSnapshot ordered by lastMessageAt	339–361
Open conversation	openConvo() — renders header, subscribes to messages, mobile slide	146–214
Send message	Encrypts, addDoc to subcollection, updates lastMessage + unread	300–336
Reactions	toggleReaction() — arrayUnion / arrayRemove on reactions.{emoji}	283–297

10. **Chef AI Chatbot** — scripts/chatbot.js

Module	What it does	Lines
Widget injection	buildChatWidget() — creates bubble + window, appends to document.body	12–42
Open/close	Bubble click toggles .hidden	48–54
Send to Gemini	Calls Gemini 2.5 Flash API, maintains conversationHistory for context	69–119
Render reply	Converts **bold** to , appends to messages div	121–142

11. **Mobile Navigation** — scripts/navbar.js 16 lines. Handles the ☰ toggle for all inner pages. Closes nav on outside click.

CSS Architecture

File	What it styles
styles/main.css	Imports 5 sub-modules for the home page
styles/auth.css	Login/Register (Bootstrap overrides + brand colors + decorative shapes)
styles/cards.css	Recipe cards, vote buttons, favourite button, feature cards
styles/recipe-details.css	Full detail page layout, hero, panels, comments

File	What it styles
styles/recipe-form.css	Create/Edit form, AI box, tabs, file upload
styles/profile.css	Profile card, avatar, inline edits, 2FA, admin rows, crop modal
styles/chat.css	Two-panel chat layout, bubbles, reactions, mobile slide transitions
styles/chatbot.css	Fixed floating bubble, chat popup window

2. Firebase Setup — Deep Dive

File: scripts/firebase.js

What it does

This file is the single entry point for all Firebase services. Every other JS file imports `auth`, `db`, or `storage` from here — none of them call `initializeApp` themselves.

```
const app = getApps().length ? getApp() : initializeApp(firebaseConfig);
```

The guard `getApps().length ? getApp() : initializeApp(...)` means the app is only initialized once even if multiple modules import this file (which happens on every page). Without this guard you'd get a "duplicate app" error in the console.

Why `initializeFirestore` instead of `getFirestore`

```
export const db = initializeFirestore(app, {
  localCache: persistentLocalCache({
    tabManager: persistentSingleTabManager()
  })
});
```

`getFirestore` gives you a plain Firestore instance. `initializeFirestore` lets you customize it — here it enables **persistent local caching**. This stores Firestore query results in the browser's **IndexedDB**, so:

- On a repeat visit, recipe data loads instantly from local storage before the network even responds
- If the user's internet drops mid-session, cached data is still visible
- `persistentSingleTabManager` means only one browser tab manages the cache at a time, preventing conflicts

Why the Firebase SDK is loaded from a CDN URL

```
import { initializeApp } from "https://www.gstatic.com/firebasejs/10.8.0/firebase-app.js";
```

Because Cookly has no build system (no webpack, no npm). It's a raw HTML/JS project, so ES module imports point directly to Google's CDN. The version `10.8.0` is pinned — if Google releases 10.9.0 this app won't automatically break.

3. Authentication — Deep Dive

File: scripts/auth.js **Used on:** pages/Login.html, pages/Register.html

The "single file, multiple forms" pattern

Both Login and Register pages load the same `auth.js` file. The script handles both forms by checking which elements exist in the DOM:

```
const loginForm = document.getElementById("loginForm"); // only on Login.html
const registerForm = document.getElementById("registerForm"); // only on Register.html
```

Then every section is guarded:

```

if (loginForm) {
  loginForm.addEventListener("submit", ...)
}
if (registerForm) {
  registerForm.addEventListener("submit", ...)
}

```

So on Login.html, registerForm is null and that branch never runs. On Register.html, loginForm is null and that branch is skipped.

Email Login flow

```

User fills email + password → loginForm.submit
→ signInWithEmailAndPassword(auth, email, password)
→ [SUCCESS]: save name to localStorage + redirect to Cookly.html
→ [ERROR: multi-factor-auth-required]: save resolver, hide loginForm, show mfaForm
→ [ERROR: other]: show "Invalid email or password"

```

The `localStorage.setItem("userName", name)` is done as a cache so that the home page navbar can show the user's name immediately before Firebase's `onAuthStateChanged` fires (which has a small delay on page load).

Why MFA uses a resolver object

When Firebase throws `auth/multi-factor-auth-required`, the error object includes a `resolver` — a special object that represents the in-progress login session. This is how Firebase pauses authentication mid-way:

```

mfaResolver = getMultiFactorResolver(auth, err); // save the paused session

```

Later, when the user enters their 6-digit code:

```

const hint      = mfaResolver.hints[0];           // the enrolled factor (TOTP)
const assertion = TotpMultiFactorGenerator.assertionForSignIn(hint.uid, code);
await mfaResolver.resolveSignIn(assertion);        // complete the login

```

The `hint.uid` is not the user's UID — it's the factor's unique ID (a short string like `abc123`). Firebase uses it to know which TOTP enrollment to verify against.

Google Sign-In

```

const provider = new GoogleAuthProvider();
const cred = await signInWithPopup(auth, provider);

```

`signInWithPopup` opens a Google OAuth window. If the user closes it, Firebase throws `auth/popup-closed-by-user` — the code catches this and does nothing (no error message, since the user chose to close it). All other errors show the generic message.

Google sign-in can also trigger MFA if the user has 2FA enrolled on their Cookly account. The catch block handles this the same way as email login.

Forgot Password

This is a hidden form inside Login.html. Three DOM elements control it:

- `forgotPasswordLink` — clicking this shows `forgotForm` and hides `loginForm`
- `backToLogin` — clicking this reverses it
- `sendResetBtn` — calls `sendPasswordResetEmail(auth, email)`

Firebase sends a reset link from noreply@cookly-e712f.firebaseio.com. The user clicks it, gets a Firebase-hosted page to enter a new password, and that's it — the app doesn't need to handle the actual password change.

Registration

```

User fills name + email + password + confirm
→ createUserWithEmailAndPassword(auth, email, password)
→ updateProfile(cred.user, { displayName: name })
→ redirect to Cookly.html

```

`createUserWithEmailAndPassword` creates the account but the `displayName` starts empty. `updateProfile` is called immediately after to set it. Both calls are awaited so the profile is ready before redirecting.

The confirmation password is checked purely client-side — Firebase only needs the final password. If they don't match, we return early before any Firebase call.

Why no email verification at registration

Email verification (`sendEmailVerification`) is not called at registration. Instead, it's enforced at the point of action — voting and commenting both check `user.emailVerified` before allowing the operation. This is intentional: users can browse and use most features without verifying.

4. Recipe Discovery — Deep Dive

File: `scripts/home.js` **Used on:** `Cookly.html`

The "ensure" pattern (lines 20–92)

The first thing `home.js` does is call `ensureNavActions()` and `ensureRecipeUI()`. These functions check whether key DOM elements like `#recipeGrid` and `#categoryDropdown` exist, and if they don't, they create and inject them. This is a safety fallback — if the HTML was loaded without these elements for some reason, the JS won't crash.

State variables

```
let currentUser      = null; // Firebase user object
let allRecipes       = [];   // full loaded dataset
let filteredRecipes  = [];   // after search/filter
let favoriteRecipeIds = new Set();
let votesCache       = new Map(); // recipeId → {upvotes, downvotes}
let userVotesCache   = new Map(); // recipeId → "up" | "down"
let currentSource     = "default";
let selectedCategory = "";
let currentSort       = "default";
let currentPage      = 1;
const PAGE_SIZE       = 12;
```

These are all module-level variables (not inside any function). They act as the page's single source of truth. When you filter, `filteredRecipes` is updated; when you load more, `currentPage` increments. The caches (`votesCache` , `userVotesCache`) avoid re-fetching vote data for cards already seen.

Boot sequence

```
onAuthStateChanged(auth, async (user) => {
  currentUser = user;
  updateNavbar(user);
  await loadFavoriteIds();
  await loadRecipes();
});
```

Everything starts here. `onAuthStateChanged` fires once when the page loads (either with a user or null), and again whenever auth state changes (login/logout). On each fire:

1. `updateNavbar` updates the UI to show/hide guest vs user buttons
2. `loadFavoriteIds` fills `favoriteRecipeIds` so heart icons render correctly
3. `loadRecipes` fetches recipes from the correct Firestore collection

The source system

`currentSource` determines which Firestore collection `loadRecipes` queries:

currentSource	Collections queried
"default"	<code>recipes</code> + <code>published_recipes</code>
"community"	<code>published_recipes</code> only
"mine"	<code>users/{uid}/recipes_created</code> (requires login)
"all"	All three above

After loading, each recipe gets a `_source` property tagged on:

```
allRecipes = snap.docs.map(d => ({ id: d.id, _source: "community", ...d.data() }));
```

This `_source` tag is used later when rendering cards — "mine" recipes don't get vote buttons since you can't vote on your own recipes.

Why votes are loaded lazily (not with recipes)

Loading vote counts for 50+ recipes upfront would mean 50+ Firestore reads every page load. Instead, votes are only fetched for the 12 cards currently visible on screen:

```
const votable = slice.filter(r => r._source !== "mine" && r.id && !votesCache.has(r.id));
if (votable.length) {
  Promise.all(votable.map(r => getDoc(doc(db, "recipe_interactions", r.id)))).then(docs => {
    // update the already-rendered card counters in-place
  });
}
```

The flow is:

1. Cards render immediately with vote counts of 0
2. A `Promise.all` batch-fetches all visible recipes' vote docs in parallel
3. When results come back, the DOM counters are updated in-place

`votesCache` ensures that scrolling "Load More" doesn't re-fetch votes for already-seen cards.

The Firestore vote transaction

Voting uses `runTransaction` so it's atomic — no race conditions if two users vote at the same time:

```
await runTransaction(db, async (tx) => {
  const voteDoc = await tx.get(voteRef); // user's personal vote
  const counterDoc = await tx.get(counterRef); // global count

  const existing = voteDoc.exists() ? voteDoc.data().type : null;

  if (existing === type) {
    // User clicked the same button again → undo (toggle off)
    tx.delete(voteRef);
    counts[...] -= 1;
  } else {
    // New vote or switching up/down
    if (existing) counts[existing...] -= 1; // remove old vote
    counts[type...] += 1;
    tx.set(voteRef, { type });
  }
  tx.set(counterRef, counts);
});
```

Toggle behavior: clicking ☒ when you already voted ☒ removes the vote. Switching from ☒ to ☐ removes the upvote and adds a downvote in one atomic operation.

Filtering pipeline

```
User input event → applyFilters()
  → filter allRecipes by: search text, category, maxTime, maxCalories
  → sort by: mostLiked / leastLiked (using votesCache) / default
  → renderRecipes(filtered) ← resets page to 1
    → renderPage()           ← renders PAGE_SIZE cards
```

All filtering is **client-side** — the full recipe list is loaded once and filtered in memory. This works because the recipe dataset is small (dozens, not thousands). The `extractNumber` helper parses values like `"30 min"` or `"350 cal"` by regex-matching the first number, making filtering work regardless of whether the Firestore value is `30`, `"30"`, or `"30 min"`.

The debounce on search

```
function debounce(fn, delay = 220) {
  let timer = null;
  return (...args) => {
    window.clearTimeout(timer);
    timer = window.setTimeout(() => fn(...args), delay);
  };
}

const handleLiveSearch = debounce(handleSearch, 220);
recipeSearch.addEventListener("input", handleLiveSearch);
```

Without debounce, every keystroke would re-filter (expensive on large lists). Debounce waits 220ms after the last keystroke before running. If you type "chicken" quickly, `applyFilters` only runs once when you stop, not 7 times.

updateNavbar and the live unread badge

```
onSnapshot(
  query(collection(db, "conversations"), where("participants", "array-contains", user.uid)),
  (snap) => {
    let total = 0;
    snap.forEach(d => { total += d.data().unread?.[user.uid] || 0; });
    // inject/remove badge span in the Messages nav link
  }
);
```

`onSnapshot` keeps a live connection to Firestore. Every time any conversation you're in gets a new unread message, this listener fires and updates the badge instantly without a page refresh. This is what makes the badge real-time.

5. Recipe Detail Page — Deep Dive

File: `scripts/recipe_details.js` **Used on:** `pages/recipe_details.html`

How the page knows which recipe to load

The URL carries everything needed:

```
recipe_details.html?id=abc123&source=community
recipe_details.html?id=xyz456&source=mine
recipe_details.html?id=def789           (no source = default recipes collection)
```

```
const params = new URLSearchParams(window.location.search);
const recipeId = params.get("id");
const source = params.get("source");
```

The `source` param determines which Firestore collection to query:

- `source=mine` → `users/{uid}/recipes_created/{id}` (requires auth, no interactions enabled)
- `source=community` → `published_recipes/{id}`
- `no source` → `recipes/{id}`

Only `mine` requires authentication. The others are public reads. Private recipes also don't get vote/comment UI — `initInteractions` is only called for community and default recipes.

The data normalization helpers

Firestore documents in the `recipes` collection were added at different times with different field naming conventions. Some have `calories`, others have `calorie`, `kcal`, or `energy`. Same problem with time: `time`, `cookTime`, `cookingTime`, `prepTime`. The normalizer handles all of them:

```
function normalizeMeta(recipe) {
  const calories = recipe.calories ?? recipe.calorie ?? recipe.kcal ?? recipe.energy ?? "N/A";
  const time = recipe.time ?? recipe.cookTime ?? recipe.cookingTime ?? recipe.prepTime ?? "N/A";
  // then append "cal" / "min" if not already present
}
```


The `??` (nullish coalescing) chains through field names until one is found. This means the same render function works for every recipe in the database regardless of how it was originally saved.

The `renderRecipe` function builds all HTML as a single string

```
detailsCard.innerHTML = `
  <div class="details-hero">...</div>
  <div class="details-content-wrap">
    ...
    <section id="interactionPanel"></section> ← placeholder
  </div>
`;
```

The entire card is rendered as one `innerHTML` assignment. Notice that `interactionPanel` is an empty `<section>` — it's just a placeholder. `initInteractions` runs after this and fills it in separately. This separation keeps rendering and interactivity concerns independent.

Why `buildIngredients` and `buildInstructions` handle both arrays and strings

Older recipes in Firestore stored `ingredients` as a plain string like `"flour, eggs, butter"`. Newer recipes (created through the app or AI) store it as an array `["1 cup flour", "2 eggs", "100g butter"]`. Both functions normalize this:

```
if (Array.isArray(ingredients) && ingredients.length) {
  return `<ul>...</ul>`; // new format
}
if (typeof ingredients === "string") {
  const items = ingredients.split(/\r?\n|,/); // old format, split by newline or comma
  return `<ul>...</ul>`;
}
```

Comments: why `loadComments` does NOT use `onSnapshot`

Votes use `runTransaction` (Firestore SDK) so they're accurate. Comments use a one-time `getDocs` — they load once on page load. New comments submitted by you are added to the DOM immediately via `list.insertBefore(newEl, list.firstChild)` without a re-read from Firestore. Other users' new comments won't appear until page refresh.

This was a deliberate simplicity trade-off. `onSnapshot` for comments would require a Firestore index on `(recipeId, createdAt)` and real-time reads, adding cost and complexity for a feature where real-time isn't critical.

The Message Author button

```
onAuthStateChanged(auth, (user) => {
  if (recipe.uid && (!user || user.uid !== recipe.uid)) {
    // inject "Message [name]" button into the vote row
    messageBtn.href = user
      ? `chat.html?uid=${recipe.uid}&name=${authorName}`
      : "Login.html";
  }
});
```

Three conditions must all be true:

1. The recipe has a `uid` field (it's a community recipe with a known author)
2. The button hasn't already been injected (guard against `onAuthStateChanged` firing twice)
3. The author isn't you (`user.uid !== recipe.uid`)

If you're not logged in, the button still shows but links to `Login.html`. If logged in, it links directly to the chat page with URL params so the conversation opens automatically.

6. Favourites — Deep Dive

File: `scripts/favourites.js` **Used on:** `pages/favourites.html`

The multi-collection resolution problem

Favourites are stored as document IDs in `users/{uid}/favorites/{recipeId}`. The ID alone doesn't tell you which collection the recipe lives in. It could be:

- A default recipe in `recipes/`
- A community recipe in `published_recipes/`
- One of your own recipes in `users/{uid}/recipes_created/`

The solution is a sequential fallback:

```
async function fetchRecipeById(recipeId, userId) {
  // Try 1: default recipes
  let snap = await getDoc(doc(db, "recipes", recipeId));
  if (snap.exists()) return { id: snap.id, ...snap.data() };

  // Try 2: community recipes
  snap = await getDoc(doc(db, "published_recipes", recipeId));
  if (snap.exists()) return { id: snap.id, ...snap.data() };

  // Try 3: user's own recipes
  if (userId) {
    snap = await getDoc(doc(db, "users", userId, "recipes_created", recipeId));
    if (snap.exists()) return { id: snap.id, ...snap.data() };
  }
  return null; // recipe was deleted or inaccessible
}
```

This means up to 3 Firestore reads per favourite. For a user with 10 favourites, that's up to 30 reads — acceptable at this scale, but worth noting.

Why unfavourite only removes the DOM node without reloading

```
async function unfavorite(recipeId, cardEl) {
  await deleteDoc(doc(db, "users", currentUser.uid, "favorites", recipeId));
  cardEl.remove(); // immediate DOM removal, no re-fetch
}
```

Re-loading all favourites after removing one would cause a noticeable flicker. Since the card already exists in the DOM, removing the DOM element is correct, instant, and doesn't need confirmation from Firestore.

7. Create & Edit Recipe — Deep Dive

Files: `scripts/create_recipe.js`, `scripts/edit_recipe.js`

The AI generation pipeline

The AI tab has two modes: photo and text. Both ultimately call the same Gemini API endpoint and parse the same JSON response.

Photo mode:

```
User selects photo → file input change event
→ FileReader reads file as base64 (toBase64())
→ API call includes: { inlineData: { mimeType, data: base64 } }
→ Gemini analyzes the image and returns recipe JSON
```

Text mode:

```
User types description → "Generate Recipe" click
→ API call includes: { text: "Make a recipe for: [description]" }
→ Gemini returns recipe JSON
```

The prompt instructs Gemini to return **only** JSON with specific fields, wrapped in a markdown code block:

```

```json
{
 "name": "...",
 "category": "...",
 "description": "...",
 "calories": ...,
 "time": ...,
 "ingredients": [...],
 "instructions": [...]
}
```

```

Parsing Gemini's response

Gemini wraps JSON in markdown code blocks even when told not to. `parseGeminiRecipe` strips the wrapper before parsing:

```

function parseGeminiRecipe(text) {
  const cleaned = text.replace(/```json\s*/i, "").replace(/```\s*$/, "").trim();
  return JSON.parse(cleaned);
}

```

If parsing fails, the catch block shows an error message and the user can try again.

Form submission and image upload

```

User clicks Submit
→ validate required fields
→ if image file selected:
  → uploadBytes(storageRef, file)
  → getDownloadURL(snapshot.ref) → imageURL
→ addDoc(collection(db, "users/{uid}/recipes_created"), {
  name, category, description, calories, time,
  ingredients: [...], // from textarea, split by newline
  instructions: [...], // from textarea, split by newline
  imageURL,
  status: "private", // starts private, user must submit for review
  createdAt: new Date()
})

```

Storage path: `recipe_images/{uid}/{timestamp}_{filename}`. The timestamp prefix prevents filename collisions.

The difference between create and edit

`edit_recipe.js` loads the existing recipe first:

```

async function loadRecipe(recipeId) {
  const snap = await getDoc(doc(db, "users", user.uid, "recipes_created", recipeId));
  // prefill all form fields
  // show existing image if present
}

```

On submit, it uses `updateDoc` instead of `addDoc`, and only uploads a new image if the user selected a new file. If no new file is selected, the existing `imageURL` is preserved.

8. User Profile — Deep Dive

File: `scripts/profile.js` **Used on:** `pages/profile.html`

Avatar upload with Cropper.js

The avatar upload involves four steps:

1. **File selection** — hidden `<input type="file">` is triggered when user clicks the avatar
2. **FileReader** — converts the selected image to a base64 data URL for the Cropper.js preview
3. **Cropper.js** — a third-party library loaded from CDN. Shows the image in a modal with a square crop box. `aspectRatio: 1` forces a circular crop. When the user confirms, `cropper.getCroppedCanvas({ width: 400, height: 400 })` produces a 400×400 canvas
4. **Upload** — `canvas.toBlob(resolve, "image/jpeg", 0.9)` converts to a JPEG blob (0.9 = 90% quality), `uploadBytes` sends to Firebase Storage, `getDownloadURL` returns the public URL, `updateProfile(user, { photoURL: url })` sets it on the auth user

The result: every profile photo is a 400×400 JPEG, regardless of what the user uploaded.

The inline edit pattern (name, email, password)

All three fields use the same UX pattern:

```
[Display text] [= pencil button]
                ↓ click pencil
[Input field] [Save] [Cancel]
```

In the DOM, each has two sibling divs: `nameDisplay` and `nameEdit`. Clicking the pencil hides `nameDisplay` and shows `nameEdit`. Save/Cancel toggle back.

The difference between them:

- **Name:** `updateProfile(user, { displayName: newName })` — immediate update on the auth user object
- **Email:** `verifyBeforeUpdateEmail(user, newEmail)` — sends a verification email to the NEW address; the email only changes after the user clicks the link. This prevents someone from locking you out by changing your email.
- **Password:** `sendPasswordResetEmail(auth, user.email)` — doesn't ask for the new password at all; Firebase handles the reset on their hosted page

2FA (TOTP) enrollment

This is the most complex flow in the entire codebase:

```
Click "Enable 2FA"
→ check email verified (if not, send verification email and stop)
→ multiFactor(user).getSession() → get an MFA session token
→ TotpMultiFactorGenerator.generateSecret(session) → get a TOTP secret
→ secret.generateQrCodeUrl(email, "Cookly") → get an otpauth:// URL
→ render QR code via api.qrserver.com → user scans with Google Auth / Authy
→ user enters 6-digit code
→ TotpMultiFactorGenerator.assertionForEnrollment(secret, code)
→ multiFactor(user).enroll(assertion, "Cookly Authenticator")
→ 2FA is active — next login requires a 6-digit code
```

`pendingTotpSecret` is a module-level variable that holds the secret between the QR display step and the verification step. If the user cancels, `pendingTotpSecret = null` discards the pending enrollment.

To disable: `multiFactor(user).unenroll(factor)` — finds the enrolled TOTP factor and removes it.

Admin gate

```
if (ADMIN_EMAILS.includes(user.email)) {
  loadAdminSection();
}
```

The admin section is in the HTML but hidden by default. `loadAdminSection` makes it visible and populates it. The check is client-side — security is enforced server-side by Firestore rules which check `request.auth.token.email`.

9. Admin Approval System — Deep Dive

File: `scripts/profile.js` lines 298–444

The two-collection publish workflow

When a user submits a recipe for review, two things happen:

1. `users/{uid}/recipes_created/{id}` → status updated to "pending"
2. A new doc is written to `pending_reviews/` containing a full copy of the recipe data plus `uid`, `recipeId`, `ownerName`, `submittedAt`

The `pending_reviews` collection is a snapshot — it holds a copy of the recipe at submission time. This means the admin sees exactly what was submitted even if the user edits the original after submission.

When admin **approves**:

```
// 1. Write full copy to public collection
await addDoc(collection(db, "published_recipes"), {
  ...recipeData, uid, recipeId, ownerName, status: "published", publishedAt: new Date()
});
// 2. Update user's recipe status
await updateDoc(doc(db, "users", uid, "recipes_created", recipeId), { status: "published" });
// 3. Remove from queue
await deleteDoc(doc(db, "pending_reviews", pendingId));
```

When admin **rejects**:

```
// 1. Revert user's recipe status to private
await updateDoc(doc(db, "users", uid, "recipes_created", recipeId), { status: "private" });
// 2. Remove from queue
await deleteDoc(doc(db, "pending_reviews", pendingId));
```

Note: rejection does NOT write to `published_recipes`. The recipe goes back to being private — the user can edit it and resubmit.

The admin shortcut

Admins skip the review queue entirely. When `total_carnage24@yahoo.com` clicks Publish on one of their recipes:

```
if (isAdmin) {
  // Skip pending_reviews entirely
  await updateDoc(doc(..., "recipes_created", recipeId), { status: "published" });
  await addDoc(collection(db, "published_recipes"), { ...recipeData, publishedAt: new Date() });
}
```

10. User-to-User Messaging — Deep Dive

File: `scripts/chat.js` **Page:** `pages/chat.html`

The deterministic conversation ID

```
function getConvoId(uid1, uid2) {
  return [uid1, uid2].sort().join("_");
}
```

UIDs are sorted alphabetically before joining. This means:

- User A (uid: "abc") messaging User B (uid: "xyz") → ID: "abc_xyz"
- User B messaging User A → same ID: "abc_xyz"

Both users independently compute the same ID without a database lookup. This eliminates the need for a "find or create" query to check if a conversation exists.

The encryption system

Messages are encrypted **in the browser** before being sent to Firestore. Anyone looking at the Firestore console only sees base64 ciphertext.

Key derivation (one-time per conversation):

```

async function getKey(convoId) {
  if (keyCache.has(convoId)) return keyCache.get(convoId); // cached

  const raw = new TextEncoder().encode(convoId + CRYPTO_PEPPER);
  const keyMaterial = await crypto.subtle.importKey("raw", raw, "PBKDF2", false, ["deriveKey"]);
  return crypto.subtle.deriveKey(
    { name: "PBKDF2", salt: new TextEncoder().encode(convoId), iterations: 100000, hash: "SHA-256" },
    keyMaterial,
    { name: "AES-GCM", length: 256 },
    false, ["encrypt", "decrypt"]
  );
}

```

PBKDF2 with 100,000 iterations is intentionally slow – this makes brute-force attacks impractical. The key is derived from `convoId + "c00k1y-e2ee-pepper-2024"`. Since both users have the same conversation ID and the same pepper (it's in the source code), they independently derive the identical key.

Encrypt:

```

const iv = crypto.getRandomValues(new Uint8Array(12)); // random 12-byte IV
const encrypted = await crypto.subtle.encrypt({ name: "AES-GCM", iv }, key, encoded);
// prepend IV to ciphertext: [12 bytes IV][N bytes ciphertext]
const combined = new Uint8Array(12 + encrypted.byteLength);
combined.set(iv, 0);
combined.set(new Uint8Array(encrypted), 12);
return btoa(String.fromCharCode(...combined)); // base64 string → stored in Firestore

```

A new random IV is generated for every message. This ensures the same message text produces different ciphertext each time. Without this, an attacker could detect repeated messages.

Decrypt:

```

const combined = Uint8Array.from(atob(ciphertext), c => c.charCodeAt(0));
const iv = combined.slice(0, 12); // first 12 bytes
const payload = combined.slice(12); // rest
return new TextDecoder().decode(
  await crypto.subtle.decrypt({ name: "AES-GCM", iv }, key, payload)
);

```

Old unencrypted messages (written before encryption was added) fail decryption gracefully – the `catch` block returns the raw text as-is.

The real-time subscription model

Conversations list:

```

convoUnsub = onSnapshot(
  query(collection(db, "conversations"),
    where("participants", "array-contains", user.uid),
    orderBy("lastMessageAt", "desc")
  ),
  (snap) => { /* rebuild convo list */ }
);

```

Messages within a conversation:

```

messagesUnsub = onSnapshot(
  query(collection(db, "conversations", convoId, "messages"),
    orderBy("createdAt", "asc")
  ),
  (snap) => { /* rebuild message thread */ }
);

```

Both use `onSnapshot` for real-time updates. When you close a conversation, `messagesUnsub()` is called to unsubscribe and stop paying for reads.

Unread counter mechanism

```
// When someone sends a message
await updateDoc(convoRef, {
  lastMessage:      encryptedText,
  lastMessageAt:    new Date(),
  [`unread.${otherUid}`]: (current.unread?.[otherUid] || 0) + 1 // increment recipient
});

// When recipient opens the conversation
await updateDoc(convoRef, {
  [`unread.${currentUser.uid}`]: 0 // reset their counter
});
```

The `unread` field is a map: `{ "uid1": 2, "uid2": 0 }`. Each user has their own counter. The home page subscribes to this and shows the sum as a badge on the Messages nav link.

Mobile layout: slide transitions

On mobile (`width ≤ 720px`), the sidebar and chat panel are both `position: absolute; inset: 0` — they each fill the full screen. The chat panel starts off-screen to the right:

```
.chat-main { transform: translateX(100%); }
.chat-sidebar { z-index: 2; }
```

When you tap a conversation:

```
function showChat() {
  chatSidebar.classList.add("hidden-mobile"); // slides sidebar left, pointer-events: none
  chatMain.classList.add("visible-mobile");   // slides panel to translateX(0), z-index: 3
}
```

CSS transitions handle the animation at 0.25s ease. The back button in the header restores the sidebar via `showSidebar()`.

11. Chef AI Chatbot — Deep Dive

File: `scripts/chatbot.js` **Loaded on:** `Cookly.html` (home page only)

Widget injection

The chatbot is not in the HTML. It creates and injects its own elements into `document.body` at runtime:

```
function buildChatWidget() {
  const bubble = document.createElement("button"); // the ☰ floating button
  const window_ = document.createElement("div");  // the chat popup
  document.body.appendChild(bubble);
  document.body.appendChild(window_);
}
```

This means the chatbot can be added/removed from pages simply by adding/removing the `<script>` tag — no HTML changes needed.

Conversation history and context

```
const conversationHistory = [];

// On each message:
conversationHistory.push({ role: "user", parts: [{ text: userMessage }] });

const response = await fetch(GEMINI_URL, {
  method: "POST",
  body: JSON.stringify({
    system_instruction: { parts: [{ text: SYSTEM_PROMPT }] },
    contents: conversationHistory // entire history sent every request
  })
});

conversationHistory.push({ role: "model", parts: [{ text: reply }] });
```

Every API call sends the **entire conversation history** so Gemini has context. If you asked "how do I make pasta?" and then ask "what sauce goes with it?", Gemini understands "it" refers to pasta. The downside is that very long conversations become expensive (more tokens sent per request).

The system prompt restriction

```
const SYSTEM_PROMPT = `You are Cookly's friendly recipe assistant. Your name is Chef AI.
YOU ONLY help with cooking, recipes, ingredients, meal planning, nutrition, and food-related topics.
If anyone asks about anything unrelated to food or cooking, politely decline...`;
```

This is sent as `system_instruction` on every request. It's not foolproof — users could potentially jailbreak it — but it's sufficient for a student project.

12. Mobile Navigation

File: `scripts/navbar.js` **Loaded on:** All inner pages (`chat.html`, `profile.html`, `favourites.html`, etc.)

The home page (`Cookly.html`) has its own mobile nav toggle inside `home.js` . Inner pages use this shared `navbar.js` :

```
menuToggle.addEventListener("click", (e) => {
  e.stopPropagation();
  navLinks.classList.toggle("active");
});

document.addEventListener("click", (e) => {
  if (!navLinks.contains(e.target) && e.target !== menuToggle) {
    navLinks.classList.remove("active");
  }
});
```

`e.stopPropagation()` on the toggle prevents the event from bubbling up to `document` , which would immediately close the menu after opening it. The document-level listener closes the menu when clicking anywhere outside it.

13. CSS Architecture

The home page CSS is split into 5 modules imported by `styles/main.css` :

```
@import "main.base.css";           /* CSS variables, body, fonts */
@import "main.header-buttons.css"; /* navbar, auth buttons */
@import "main.hero-favorites.css"; /* hero section, decorative shapes */
@import "main.content.css";        /* search, filters, recipe grid */
@import "main.responsive.css";     /* all mobile breakpoints */
```

This separation means you can find the right CSS quickly: if the grid looks wrong, look in `main.content.css` . If the navbar button is broken, look in `main.header-buttons.css` .

CSS custom properties (defined in `main.base.css`) are used throughout:


```

:root {
  --primary: #022175; /* dark blue */
  --accent: #e4614a; /* coral/orange */
  --bg: #f5eede; /* warm cream */
  --white: #ffffff;
  --muted: #6b7280;
  --border: #e5e7eb;
  --shadow: 0 14px 40px rgba(2, 33, 117, 0.1);
  --radius: 16px;
}

```

Every file references these variables. Changing `--primary` from blue to green would cascade across the entire site.

14. Firestore Data Model

```

recipes/                                ← built-in recipe library (read-only by clients)
  {recipeId}/
    name, category, description, calories, time, imageURL, ingredients[], instructions[]

published_recipes/                      ← community-approved recipes (public read)
  {auto-id}/
    ...same fields as recipes...
    uid, recipeId, ownerName, status:"published", publishedAt

users/
  {uid}/
    favorites/
      {recipeId}/
        recipeId, createdAt
    recipes_created/
      {recipeId}/
        ...recipe fields...
        status: "private" | "pending" | "published"
        createdAt

pending_reviews/                        ← admin approval queue
  {auto-id}/
    ...recipe fields...
    uid, recipeId, ownerName, submittedAt

recipe_interactions/
  {recipeId}/
    upvotes: Number, downvotes: Number
    votes/
      {uid}/
        type: "up" | "down"
    comments/
      {auto-id}/
        userId, userName, userPhotoURL, text, createdAt

conversations/
  {uid1_uid2}/
    participants: [uid1, uid2]
    participantInfo: { uid1: {name, photoURL}, uid2: {name, photoURL} }
    lastMessage: String (encrypted)
    lastMessageAt: Timestamp
    unread: { uid1: Number, uid2: Number }
    messages/
      {auto-id}/
        senderId, senderName, text (encrypted), createdAt, reactions: {}

```

15. How Everything Connects

This section traces the full lifecycle of three key user journeys.

Journey 1: User registers and logs in

```
Register.html
→ auth.js: createUserWithEmailAndPassword + updateProfile
→ localStorage: userName cached
→ redirect → Cookly.html

Cookly.html loads
→ home.js: onAuthStateChanged fires (user is now logged in)
→ updateNavbar(user): hide "Log In / Get Started", show "Hi, [name] / Log Out"
→ loadFavoriteIds(): read users/{uid}/favorites → populate favoriteRecipeIds Set
→ loadRecipes(): fetch recipes + published_recipes
→ renderPage(): render 12 recipe cards
→ lazy fetch votes for these 12 in parallel
→ update card counters when votes arrive
```

Journey 2: User creates and publishes a recipe

```
profile.html → My Recipes → Create New
→ create_recipe.html
→ create_recipe.js: boot → onAuthStateChanged → redirect if not logged in
→ User optionally generates via AI:
  → Gemini API call (image or text)
  → parseGeminiRecipe() → fillFormFromRecipe()
→ User submits form:
  → uploadBytes to Firebase Storage → getDownloadURL
  → addDoc to users/{uid}/recipes_created with status:"private"
  → redirect to profile.html

profile.html
→ profile.js: loadProfile() → getDocs(users/{uid}/recipes_created)
→ buildPreviewCard() renders card with "Publish" button
→ User clicks "Publish":
  → submitForReview():
    → if admin: directly writes to published_recipes + status:"published"
    → if normal user: writes to pending_reviews + status:"pending"

admin's profile.html
→ loadAdminSection() → getDocs(pending_reviews)
→ buildPendingRow(): shows card with Approve/Reject
→ admin clicks Approve:
  → handleApprove():
    → addDoc to published_recipes
    → updateDoc users/{uid}/recipes_created → status:"published"
    → deleteDoc from pending_reviews
    → card fades out with "Approved!" message

published_recipes collection now has the recipe
→ visible on home page under "Community Recipes" source filter
→ visible on recipe_details.html with vote/comment UI
```

Journey 3: User messages a recipe author

recipe_details.html (community recipe by Heckly)

- recipe_details.js: loadRecipeDetails() → getDoc(published_recipes/{id})
- renderRecipe(recipe): render the full page
- initInteractions(recipeId, recipe):
 - onAuthStateChanged fires
 - recipe.uid exists + user.uid !== recipe.uid
 - inject "Message Heckly" button with href:
"chat.html?uid={hecklyUid}&name=Heckly"

User clicks "Message Heckly"

- chat.html?uid={hecklyUid}&name=Heckly loads

chat.js boots

- onAuthStateChanged fires
- show empty state (desktop) or keep hidden (mobile)
- loadConversations(user): onSnapshot fires
 - Heckly conversation appears in sidebar (or is created if first time)
- URL params: targetUid = hecklyUid, targetName = "Heckly"
 - getOrCreateConvo(hecklyUid, "Heckly"):
 - convoId = sort([myUid, hecklyUid]).join("_")
 - getDoc(conversations/{convoId}) → not found
 - setDoc: create conversation doc with participants, participantInfo, unread: {0, 0}
- openConvo(convoId, data):
 - chatWindow.classList.remove("hidden")
 - showChat(): slide chat-main in (mobile)
 - chatHeader.innerHTML: show back button + Heckly avatar
 - onSnapshot(messages): live stream of messages

User sends "Hey nice recipe!"

- sendMessage():
 - encryptMessage("Hey nice recipe!", convoId)
 - getKey(convoId): PBKDF2 → AES-GCM key (cached)
 - generate random IV → AES-GCM encrypt → base64 string
 - addDoc(messages/{auto-id}): { senderId, text: base64, createdAt }
 - updateDoc(conversations/{convoId}): lastMessage = base64, unread.hecklyUid += 1

Heckly's home page (if open)

- updateNavbar's onSnapshot fires
 - unread count incremented
 - Messages badge shows "1"

Heckly opens chat

- openConvo fires → unread.hecklyUid reset to 0
- messages onSnapshot loads the message
 - buildMessageEl() renders bubble with "..." placeholder
 - decryptMessage(base64, convoId) → "Hey nice recipe!"
 - bubble.innerHTML updated with decrypted text